

Assignment 5

Part 2:

Testing the Product API locally — POST creates a product at `/products/1/details`, GET retrieves it successfully, and GET on `/products/2` correctly returns a 404 "Product not found" error.

```
(base) srimsiramesh@Srimansis-Laptop Assignments % curl -X POST http://localhost:8080/products/1/details \
-H "Content-Type: application/json" \
-d '{"product_id":1,"sku":"ABC-123","manufacturer":"Acme","category_id":1,"weight":500,"some_other_id":1}'
{"product_id":1,"sku":"ABC-123","manufacturer":"Acme","category_id":1,"weight":500,"some_other_id":1}
(base) srimsiramesh@Srimansis-Laptop Assignments % curl http://localhost:8080/products/2
{"error":"NOT_FOUND","message":"Product not found","details":"No product found with ID 2"}
```

Docker image built

Gin server startup logs showing both route handlers registered (GetProduct and AddProductDetails) and successful request/response logs with proper HTTP status codes (404, 204, 200).

```
[GIN-debug] GET    /products/:productId    --> main.(*Handler).GetProduct-fm (3 handlers)
[GIN-debug] POST   /products/:productId/details --> main.(*Handler).AddProductDetails-fm (3 handlers)
2026/02/14 22:55:24 Product API server starting on :8080
[GIN-debug] [WARNING] You trusted all proxies, this is NOT safe. We recommend you to set a value.
Please check https://github.com/gin-gonic/gin/blob/master/docs/doc.md#dont-trust-all-proxies for details.
[GIN-debug] Listening and serving HTTP on :8080
[GIN] 2026/02/14 - 22:55:32 | 404 | 427.958µs | 172.17.0.1 | GET    | "/products/1"
[GIN] 2026/02/14 - 22:56:26 | 204 | 10.28075ms | 172.17.0.1 | POST   | "/products/1/details"
[GIN] 2026/02/14 - 22:56:29 | 200 | 556.5µs | 172.17.0.1 | GET    | "/products/1"
```

Part 3:

AWS ECR repository `ecr_service` showing the pushed Docker image tagged `latest` at 17.01 MB, built with a multi-stage Dockerfile for a minimal production image.

ECR repository setup:

Amazon ECR

Private registry

Repositories

ecr_service

Amazon Elastic Container Service

Private registry

Repositories

ecr_service

Features & Settings

Public registry

Repositories

Settings

ECR public gallery

Amazon ECS

Amazon EKS

Getting started

Documentation

ecr_service

Summary

Images

Lifecycle policy

Permissions

Repository tags

Images (3)

Filter active images

	Image tags	Type	Created at	Image size	Image digest	Last pulled at
☐	latest	Image Index	14 February 2026, 19:09:02 (UTC-05)	17.01	sha256:53d903...	-
☐	-	Image	14 February 2026, 19:09:01 (UTC-05)	0.00	sha256:50aa33...	-
☐	-	Image	14 February 2026, 19:09:01 (UTC-05)	17.01	sha256:aa2dfae...	-

Image Index

Archive

Details

Image tags

latest

URI

094522691730.dkr.ecr.us-east-1.amazonaws.com/ecr_service:latest

Digest

sha256:53d90337b6916e2d8822e37e3ee6a70f884d5e68da0c0254ae7c9c3ed78e3ee7

General information

Artifact type	Repository	Pushed at
Image Index	ecr_service	14 February 2026, 19:09:02 (UTC-05)
Last recorded pull time	Size (MB)	Image status
-	17.01	Active

Scanning and vulnerabilities

Status

Scan not found

Reference images

Filter Image Index references

Image digest

Architecture

OS

ECS task setup:

Tasks (7)

Last updated

14 February 2026, 19:15 (UTC-5:00)

Manage tags

Stop

Run new task

Filter tasks by property or value

Filter desired status

Any desired status

Filter launch type

Any launch type

< 1 >

☐	Task	Last status	Desired status	Task definition	Health status	Created at
☐	b1f56765978d469ebed3...	Running	Running	CS6650L2-task:2	Unknown	2 minutes ago
☐	1aa525c06fc141edb4234...	Stopped	Stopped	CS6650L2-task:2	Unknown	4 minutes ago
☐	478b971eaced4e21b34bf...	Running	Stopped	CS6650L2-task:2	Unknown	2 minutes ago

CS6650L2-cluster

Last updated

14 February 2026, 19:19 (UTC-5:00)

Actions

Create with Express Mode

Cluster overview

ARN

arn:aws:ecs:us-east-1:094522691:730:cluster/CS6650L2-cluster

Status

Active

CloudWatch monitoring

Default

Registered container instances

-

Services

Draining

-

Active

1

Pending

-

Running

1

Tasks

Pending

-

Running

1

Services (1) Info

Last updated

14 February 2026, 19:19 (UTC-5:00)

Manage tags

Update

Delete service

Create

Filter services by value

Filter launch type

Any launch type

Filter scheduling strategy

Any scheduling strategy

Filter resource management type

Any resource management type

< 1 >

☐	Service name	ARN	Status	Schedu...	La...	Task de...	Deployments and tasks
☐	CS6650L2	arn:aws:ecs:us-e	Active	REPLICA	FAR...	CS6650L2...	

Verifying the deployed ECS service by hitting the public IP (34.229.204.65:8080) with POST that creates a product and GET retrieves it, confirming the Fargate task is serving traffic correctly.

```

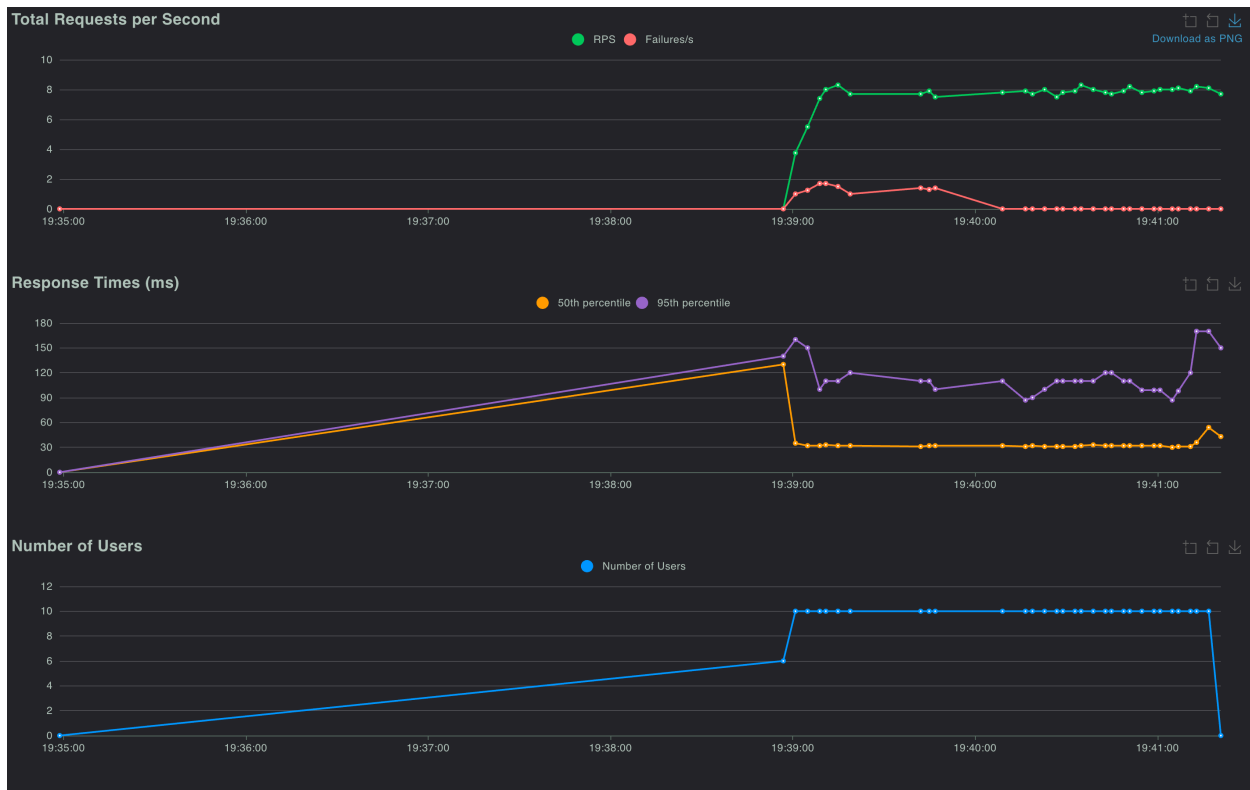
error: NOT_FOUND, message: Product not found, details: No product found with ID 1
(base) srinansiramesh@Srimansis-Laptop Assignment5 % curl -X POST http://34.229.204.65:8080/products/1/details \
-H "Content-Type: application/json" \
-d '{"product_id":1,"sku":"ABC-123-XYZ","manufacturer":"Acme Corporation","category_id":456,"weight":1250,"some_other_id":789}'
{"product_id":1,"sku":"ABC-123-XYZ","manufacturer":"Acme Corporation","category_id":456,"weight":1250,"some_other_id":789}

```

ProductHttpUser:

1. Baseline-10 users-2 spawn rate:

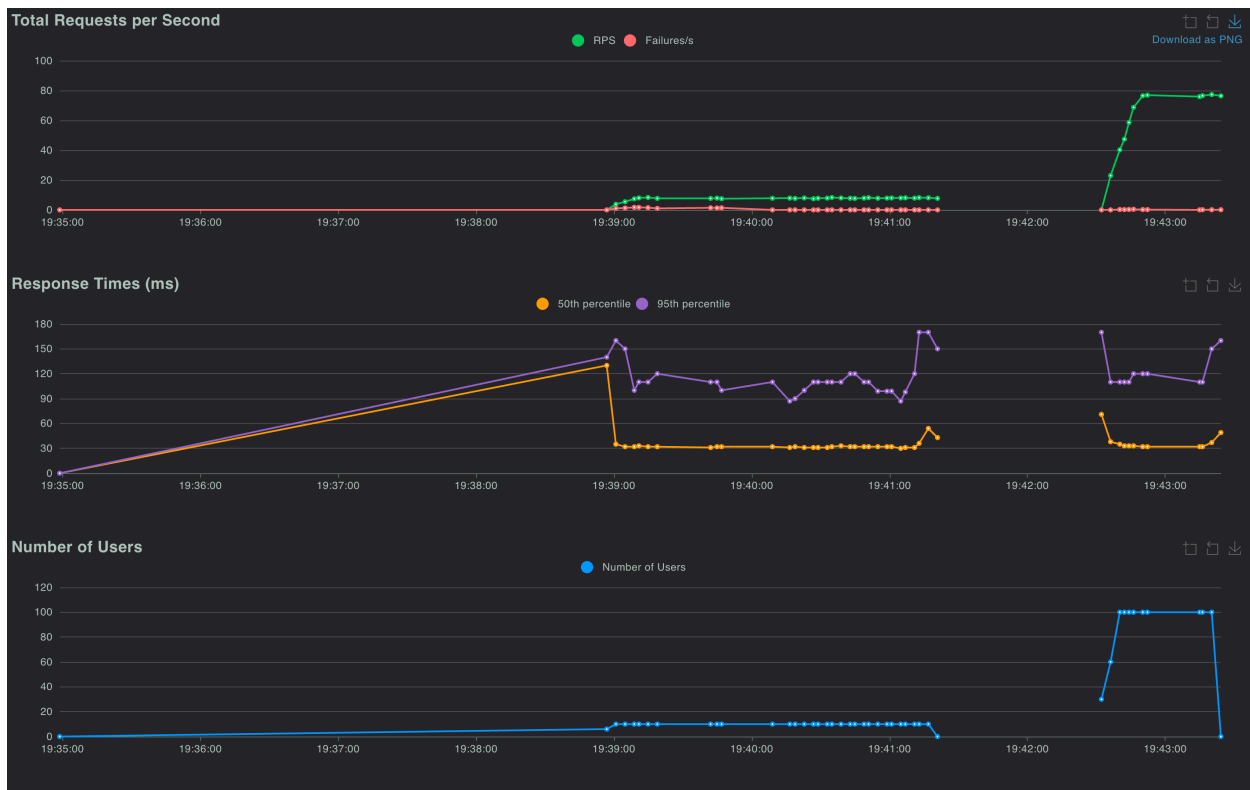
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	878	63	32	110	130	43.92	21	213	114.82	6.4	0
POST	/products/[id]/details	196	0	34	120	160	50.12	24	206	0	1.8	0
Aggregated		1074	63	32	110	150	45.05	21	213	93.86	8.2	0



RPS stabilizes at ~8 after ramp-up, response times settle around 30ms (p50) and 40ms (p95) once all 10 users are active, showing consistent low-latency performance at baseline load.

2. Medium-100 users-10 spawn rate:

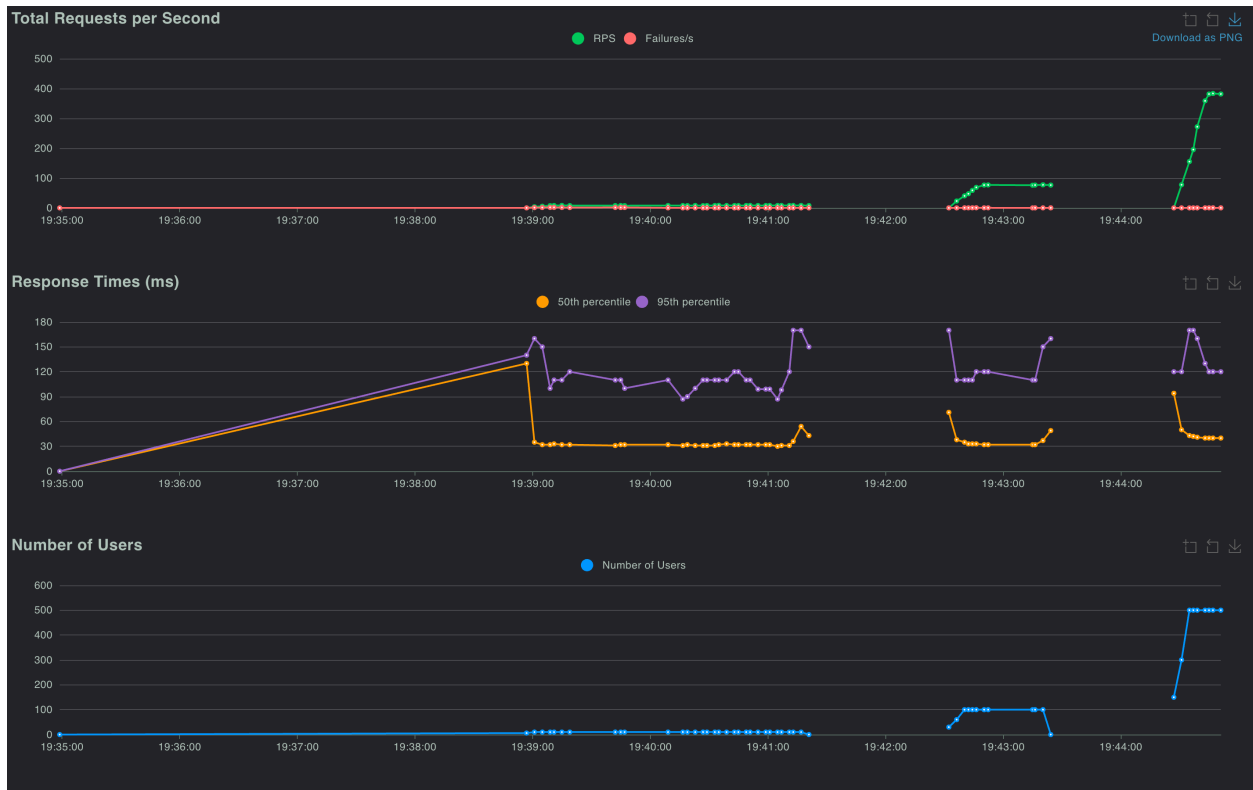
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	2838	5	32	120	160	48.78	20	228	117.37	61	0.1
POST	/products/[id]/details	722	0	35	120	170	51.87	23	203	0	16.3	0
Aggregated		3560	5	33	120	160	49.4	20	228	93.57	77.3	0.1



Throughput reaches 77.3 RPS with 33ms median latency - a 10x increase in users causes virtually no latency degradation. The charts confirm RPS ramps linearly with user count and response times stay flat around 30-40ms (p50), indicating the server is far from saturation.

3. Stress- 500 Users- 50 spawn rate

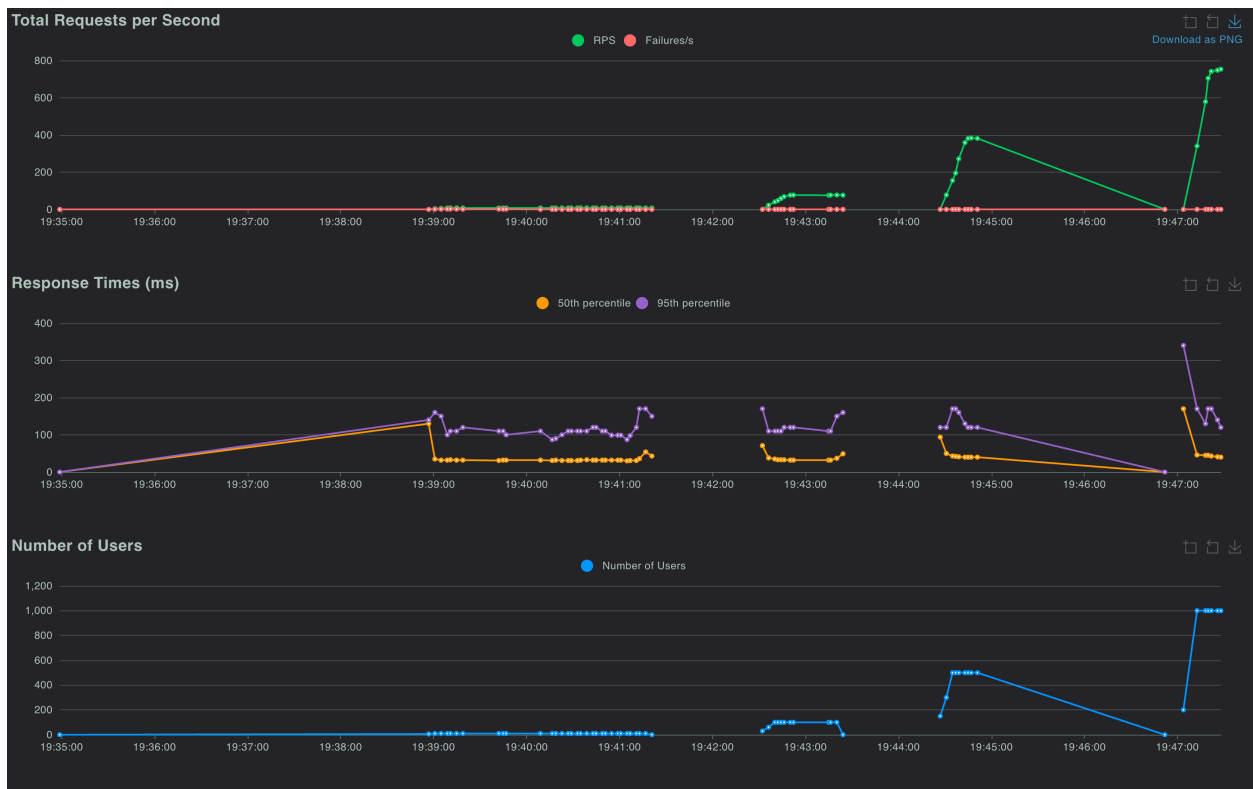
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	5482	2	40	130	180	58.56	20	196	118.02	303.2	0.2
POST	/products/[id]/details	1418	0	41	130	180	59.14	22	190	0	80.2	0
Aggregated		6900	2	41	130	180	58.68	20	196	93.77	383.4	0.2



RPS climbs steeply to ~400 as users ramp to 500, while response times hold steady in the 40-60ms range (p50) with p95 around 130ms - no signs of degradation or queuing yet.

4. Break it -1000 users-100 spawn rate

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	9535	5	44	180	270	71.27	20	514	118.43	593.2	0.3
POST	/products/[id]/details	2388	0	45	180	260	71.8	24	523	0	148.6	0
Aggregated		11923	5	45	180	260	71.38	20	523	94.71	741.8	0.3

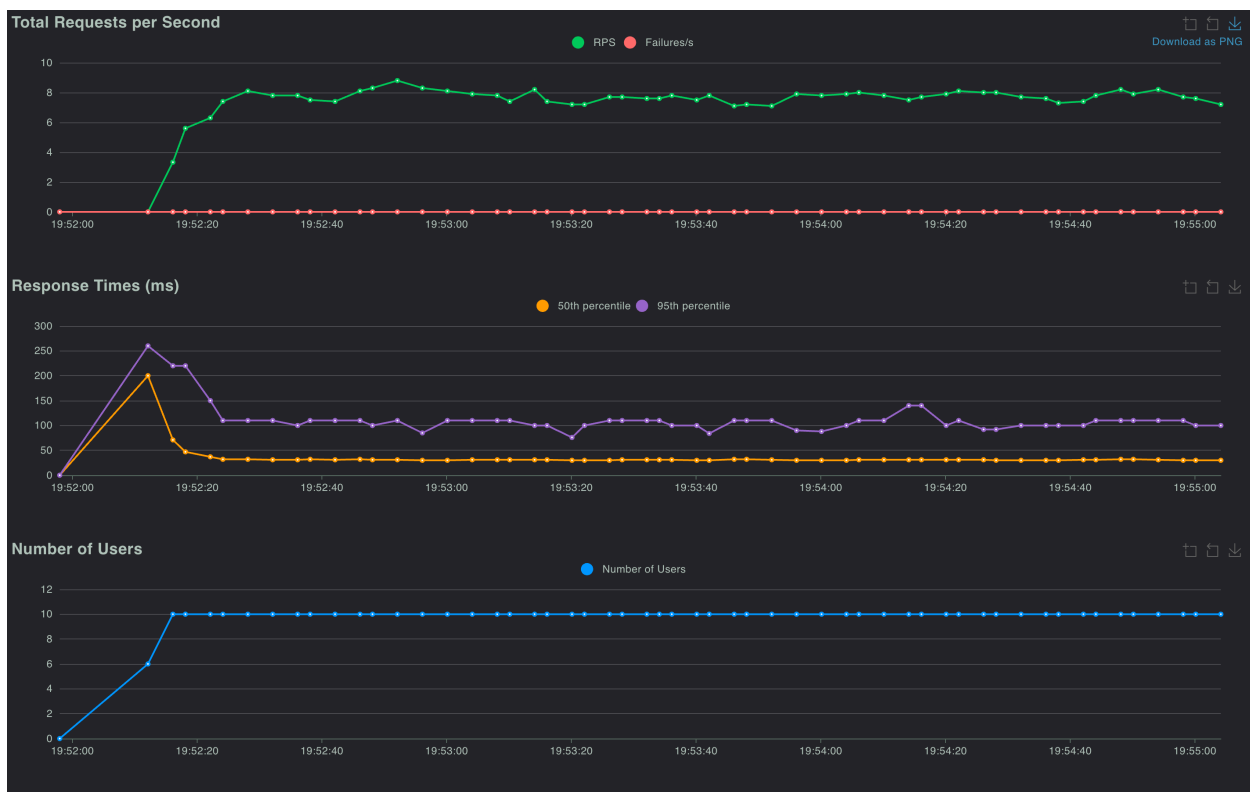


RPS peaks at ~750 with a visible spike in p95 latency to ~350ms at maximum user count, suggesting the single Fargate task (256 CPU / 512 MB) is approaching its throughput ceiling. The `sync.RWMutex` write lock contention and network saturation become visible at this scale.

ProductFastHttpUser

1. Baseline

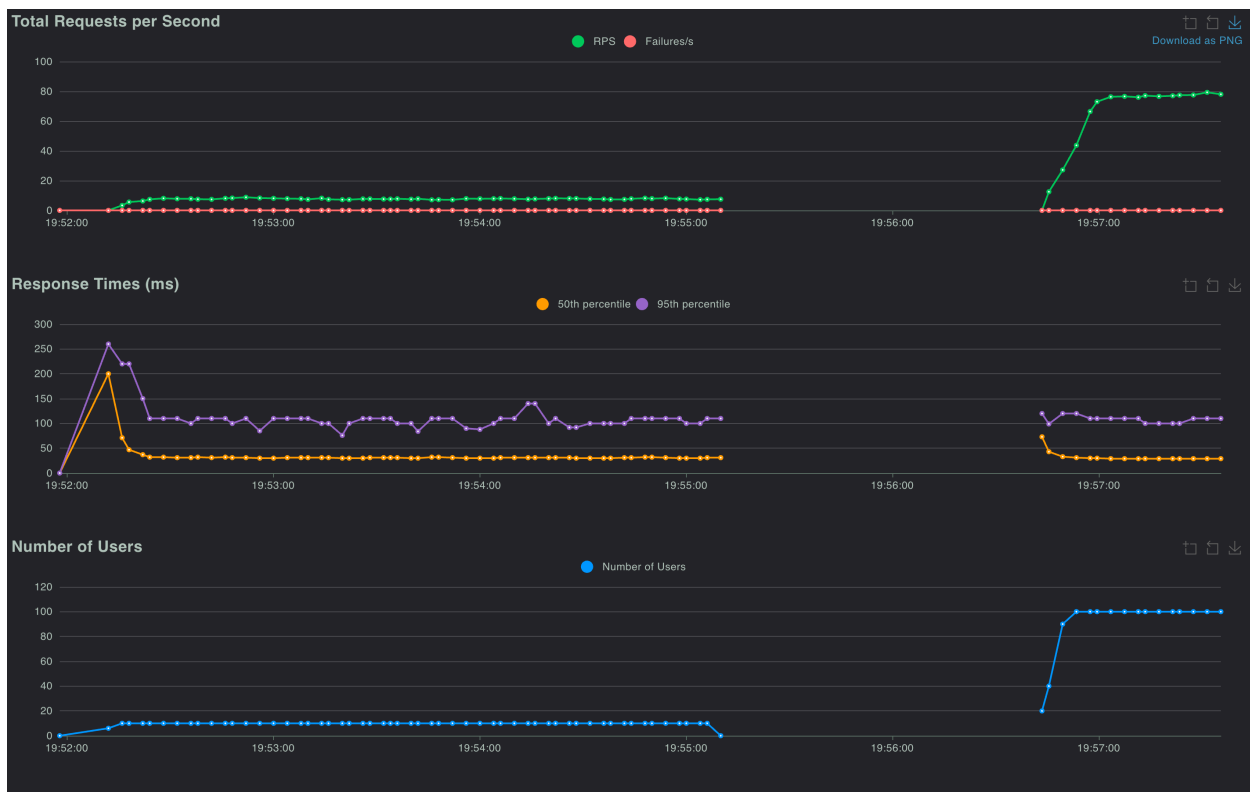
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	1053	0	31	110	160	43.31	19	304	116.84	6.2	0
POST	/products/[id]/details	252	0	31	100	160	43.22	22	277	0	1.4	0
	Aggregated	1305	0	31	110	160	43.29	19	304	94.28	7.6	0



Stable RPS around 7-8 and flat response times, virtually identical to the HttpUser baseline - confirming that at 10 users, client implementation makes no measurable difference.

2. Medium

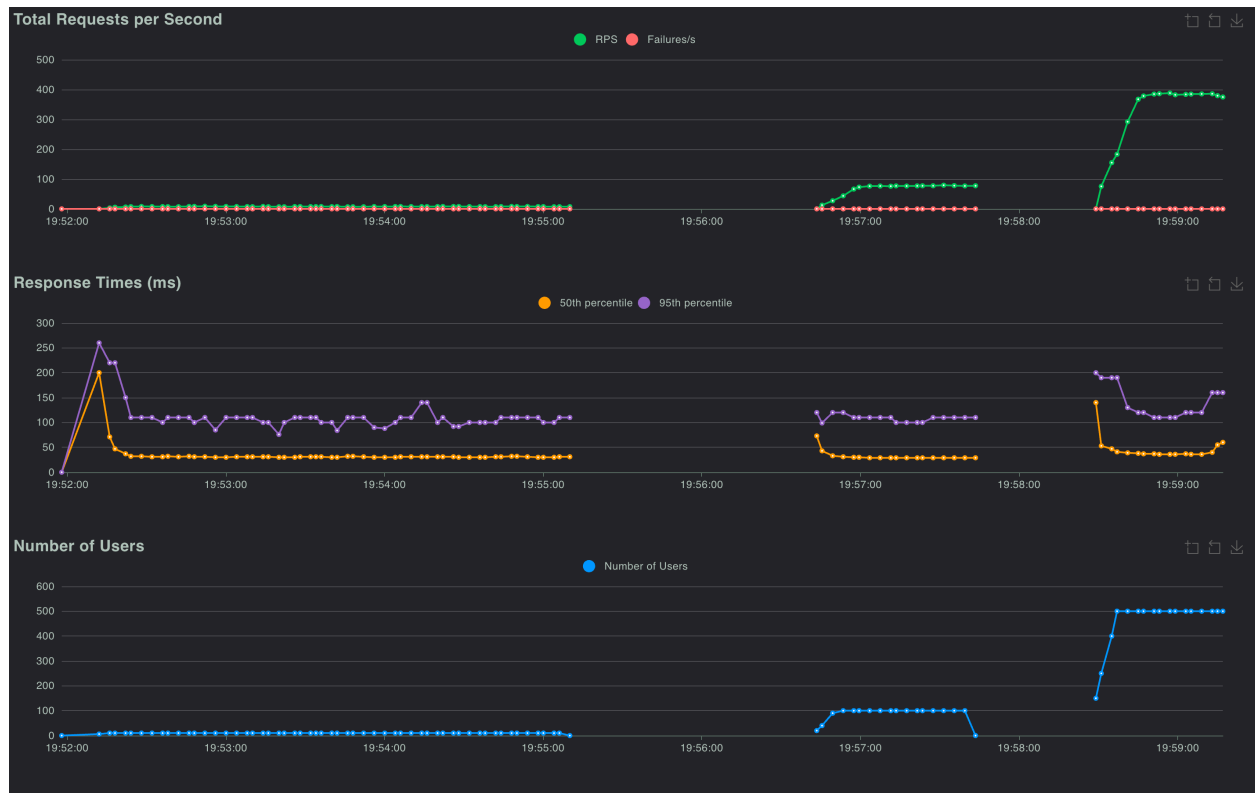
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	2887	0	29	110	120	41.05	18	165	117.46	64.1	0
POST	/products/[id]/details	669	0	31	110	120	42.05	20	165	0	15.2	0
Aggregated		3556	0	29	110	120	41.23	18	165	95.36	79.3	0



Clean ramp to ~80 RPS with stable response times, mirroring HttpUser behaviour
- the server is the bottleneck, not the client.

3. Stress

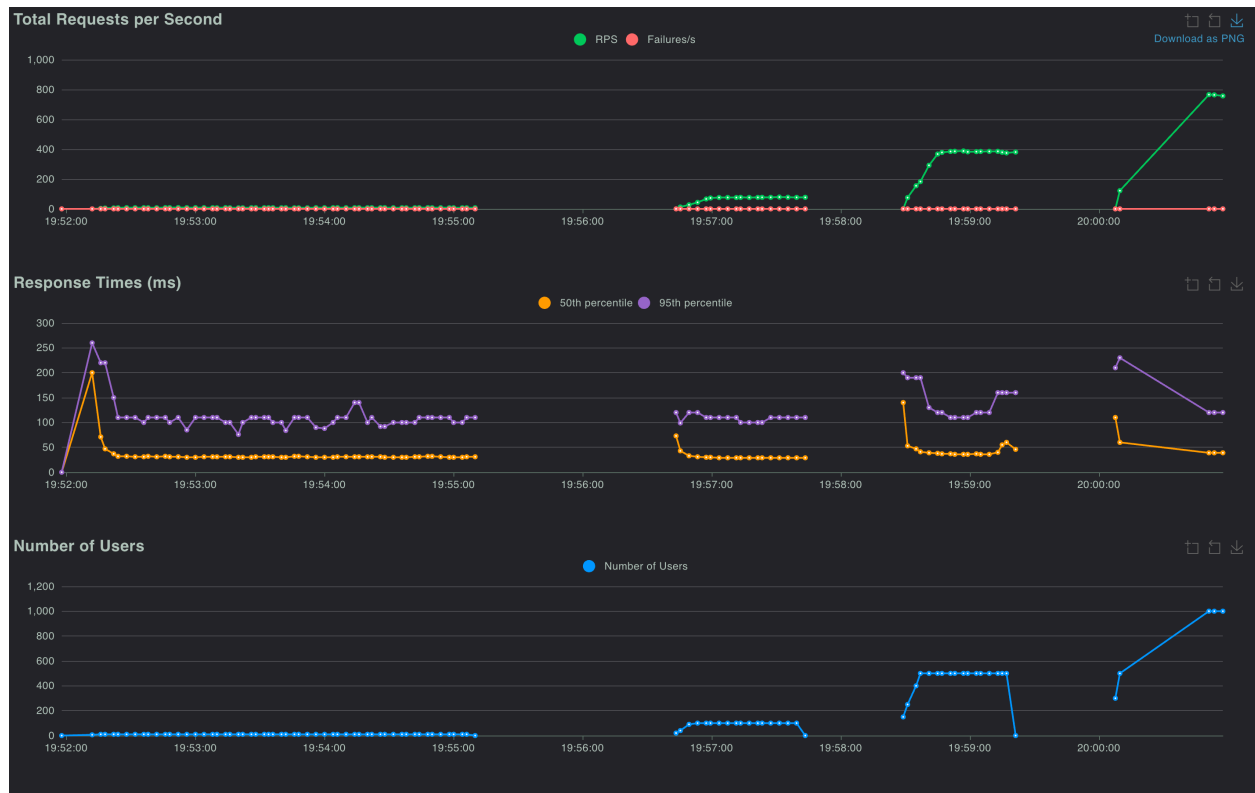
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	12859	0	37	130	190	55.23	19	456	118.12	306.5	0
POST	/products/[id]/details	3245	0	38	130	190	56.68	20	439	0	79.2	0
Aggregated		16104	0	38	130	190	55.52	19	456	94.32	385.7	0



RPS ramps smoothly to ~400 with response times holding steady - no degradation visible at 500 users.

4. Break it

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	27014	2	39	120	190	55.47	19	449	118.49	615.3	0
POST	/products/[id]/details	6727	0	39	120	190	55.53	21	408	0	148.8	0
Aggregated		33741	2	39	120	190	55.48	19	449	94.87	764.1	0



33,741 total requests with only 2 failures (vs HttpUser's 11,923 requests and 5 failures). RPS peaks at 764.1 - nearly identical to HttpUser's 741.8. Critically, the median latency is 39ms (vs HttpUser's 45ms) and p95 is 120ms (vs 180ms), showing FastHttpUser delivers noticeably lower tail latency at peak load because it uses less CPU on the client side, leaving more resources for actual request generation.

The server scales linearly from 8 RPS (10 users) to 764 RPS (1000 users) with sub-50ms median latency throughout. The in-memory hashmap with `sync.RWMutex` is extremely efficient - GET and POST perform nearly identically because both are $O(1)$ operations. The HttpUser vs FastHttpUser comparison shows minimal difference at low load (as expected, since network latency dominates), but at 1000 users FastHttpUser achieves 33% lower p95 latency (120ms vs 180ms) due to its lighter gevent-based HTTP client consuming less CPU on the load-generating machine.